# Test Plan - AI-Powered LMS

## Part I. Overall Test Plan Strategies

Our AI-powered Learning Management System employs a multi-layered testing strategy to ensure reliability, security, and performance across three independent microservices (Spring Boot backend, FastAPI AI service, and Next.js frontend). We will conduct **blackbox functional testing** to validate user-facing features such as authentication, role-based routing, and AI tutoring responses against system requirements. **Whitebox unit testing** will be performed during development to verify individual components like the Qdrant vector store wrapper, PDF parser, and backend API client within the AI service. **Integration testing** is critical given our hybrid architecture—we must ensure secure token propagation from frontend through the AI service to the backend, proper CORS configuration, and correct data flow during RAG (Retrieval-Augmented Generation) operations. **Performance testing** will validate that AI responses stream within acceptable latency (target: <3 seconds for first token, <30 seconds total response time) and that vector similarity searches complete within sub-second timeframes. **Abnormal and boundary testing** will verify graceful error handling for scenarios like invalid file uploads, missing authentication tokens, malformed course IDs, and Qdrant connection failures. Given that security is our paramount concern—with all user data access routed through the backend—we will extensively test authorization boundaries to ensure the AI service cannot bypass authentication rules.

Testing will be conducted across three environments: local development (manual service startup), containerized Qdrant instances, and integrated multi-service scenarios. For blackbox testing, we will use Swagger UI (`/docs`) endpoints for the AI service, manual frontend navigation for UI workflows, and Postman/curl for backend API validation. Whitebox testing will leverage pytest for Python components with async support, and potential JUnit integration for Java controllers (currently pending implementation). Test data includes sample course materials (text files, PDFs), mock user accounts with STUDENT and TEACHER roles, and synthetic queries for RAG evaluation. Success criteria include: all authentication flows correctly redirecting based on role, AI responses containing valid citations from ingested materials, no privilege escalation vulnerabilities, avatar uploads persisting to correct filesystem paths.

Our test case matrix covers normal operations (successful login, document ingestion, quiz generation), abnormal inputs (malformed PDFs, expired tokens, oversized files), boundary conditions (maximum chunk sizes, empty search results, concurrent user sessions), and performance benchmarks (embedding generation batching, streaming response latency). Integration test scenarios validate the complete data flow: user uploads course material → AI service authenticates with backend → PDF parsed and chunked → embeddings generated → vectors stored in Qdrant → student asks question → semantic search retrieves context → Gemini generates answer → response streams to frontend with citations. Each test case

specifies expected HTTP status codes, response payloads, database state changes, and user-visible outcomes to ensure comprehensive coverage of our hybrid microservice architecture.

---

# Part II. Test Case Descriptions

## Test Case 1: User Authentication - Student Login

- **Identifier**: AUTH-001
- **Purpose**: Verify that students can log in successfully and are redirected to the student dashboard
- **Description**: Test the complete authentication flow from login form submission to localStorage role storage and dashboard redirection
- **Inputs**:
  - Email: `student@test.com`
  - Password: `password123`
  - Role: STUDENT (stored in database)
- **Expected Outputs**:
  - HTTP 200 response from `/api/login`
  - Response body: `{id: "abc123", email: "student@test.com", role: "STUDENT"}`
  - localStorage contains: `role="STUDENT"`, `userId="abc123"`
  - Browser redirects to `/student-fe`
- **Test Type**: Normal, Blackbox, Functional, Integration

---

## Test Case 2: User Authentication - Teacher Role Routing

- **Identifier**: AUTH-002
- **Purpose**: Verify that teachers are routed to the tutor dashboard after login
- **Description**: Ensure role-based routing logic correctly interprets TEACHER role and navigates to appropriate interface
- **Inputs**:
  - Email: `teacher@test.com`
  - Password: `teacherpass`
  - Role: TEACHER (stored in database)
- **Expected Outputs**:
  - HTTP 200 response from `/api/login`
  - Response body contains `role: "TEACHER"`
  - Browser redirects to `/tutor-fe`

    - Tutor-specific sidebar menu renders (Assignments, Attendance, Classes)
- **Test Type**: Normal, Blackbox, Functional, Integration

---

## Test Case 3: Authentication Failure - Invalid Credentials

- **Identifier**: AUTH-003
- **Purpose**: Verify system rejects invalid login attempts gracefully
- **Description**: Test error handling when user provides incorrect password or non-existent email
- **Inputs**:
    - Email: `invalid@test.com`
    - Password: `wrongpassword`
- **Expected Outputs**:
    - HTTP 401 Unauthorized response
    - Response body: `{message: "Invalid credentials"}`
    - Frontend displays error alert
    - User remains on login page
    - No localStorage entries created
- **Test Type**: Abnormal, Blackbox, Functional, Integration

---

## Test Case 4: User Profile - Avatar Upload

- **Identifier**: PROFILE-001
- **Purpose**: Verify avatar image upload and filesystem persistence
- **Description**: Test multipart file upload, server-side storage in `uploads/avatars/`, and database path update
- **Inputs**:
    - User ID: `user123`
    - File: `avatar.jpg` (valid JPEG, 2MB)
    - Endpoint: `POST /api/users/user123/avatar`
- **Expected Outputs**:
    - HTTP 200 response
    - File saved to `uploads/avatars/user123_avatar.jpg`
    - Database `avatarUrl` field updated to `/uploads/avatars/user123_avatar.jpg`
    - Frontend displays new avatar at `http://localhost:8080/uploads/avatars/user123_avatar.jpg`
- **Test Type**: Normal, Blackbox, Functional, Integration

---

## Test Case 5: User Profile - Invalid File Type Upload

- **Identifier**: PROFILE-002
- **Purpose**: Verify system rejects non-image files during avatar upload
- **Description**: Test error handling when user attempts to upload PDF, TXT, or other non-image files
- **Inputs**:
  - User ID: `user123`
  - File: `document.pdf` (invalid file type)
  - Endpoint: `POST /api/users/user123/avatar`
- **Expected Outputs**:
  - HTTP 400 Bad Request or HTTP 415 Unsupported Media Type
  - Error message: "Invalid file type. Only images are allowed."
  - No file saved to filesystem
  - Database `avatarUrl` field unchanged
  - Frontend displays error message
- **Test Type**: Abnormal, Blackbox, Functional, Integration

---

## Test Case 6: AI Service - Document Ingestion (Text File)

- **Identifier**: AI-INGEST-001
- **Purpose**: Verify the AI service can ingest and vectorize course materials
- **Description**: Test complete ingestion pipeline: file parsing → chunking (512 tokens, 128 overlap) → embedding generation → Qdrant storage
- **Inputs**:
  - File path: `sample_data/intro_to_algorithms.txt`
  - Course ID: `CS101`
  - Course name: `Introduction to Algorithms`
  - Endpoint: `POST /api/ingest`
- **Expected Outputs**:
  - HTTP 200 response
  - Response body: `{status: "success", chunks_processed: 45, collection: "CS101"}`
  - Qdrant collection `CS101` created with 45 vectors
  - Each vector contains metadata: `{course_id, course_name, chunk_index, text_content}`
- **Test Type**: Normal, Blackbox, Functional, Integration

---

## Test Case 7: AI Service - RAG Query with Valid Results

- **Identifier**: AI-RAG-001
- **Purpose**: Verify semantic search retrieves relevant context and generates accurate response
- **Description**: Test end-to-end RAG flow: user query → vector search → context retrieval → Gemini generation → streaming response
- **Inputs**:
  - Query: "What is a binary search tree?"
  - Course ID: `CS101`
  - Collection: `CS101` (pre-populated with algorithm textbook)
  - Endpoint: `POST /api/chat`
- **Expected Outputs**:
  - HTTP 200 response with streaming (SSE or chunked transfer)
  - Response contains definition of binary search tree
  - Citations included: e.g., "(Source: Chapter 3, Page 42)"
  - Response streams within 3 seconds for first token
  - Complete response delivered within 30 seconds
  - Context snippets from Qdrant match query semantically
- **Test Type**: Normal, Blackbox, Functional, Integration

---

## Test Case 8: AI Service - RAG Query with No Results

- **Identifier**: AI-RAG-002
- **Purpose**: Verify graceful handling when vector search returns no relevant context
- **Description**: Test AI response when user asks about topic not covered in ingested materials
- **Inputs**:
  - Query: "Explain quantum computing algorithms"
  - Course ID: `CS101` (only contains classical algorithms)
  - Collection: `CS101`
- **Expected Outputs**:
  - HTTP 200 response (not an error)
  - Response: "I don't have information about that topic in the course materials."
  - No hallucinated information generated
  - Empty or low-relevance context from Qdrant
- **Test Type**: Boundary, Blackbox, Functional, Integration

---

## Test Case 9: AI Service - Backend Authentication Integration

- **Identifier**: AI-SECURITY-001
- **Purpose**: Verify AI service forwards user tokens to backend and respects authorization
- **Description**: Test security boundary: AI service must call backend API with user JWT to fetch assignment data
- **Inputs**:
  - Assignment ID: `HW01`
  - User token: Valid JWT for student enrolled in CS101
  - AI service calls: `backend_api.get_assignment("HW01", auth_token)`
- **Expected Outputs**:
  - AI service sends request to `http://localhost:8080/api/assignments/HW01`
  - Request includes header: `Authorization: Bearer <token>`
  - Backend validates token and returns assignment data
  - AI service receives assignment and processes it
- **Test Type**: Normal, Whitebox, Functional, Integration

---

## Test Case 10: AI Service - Backend Authorization Failure

- **Identifier**: AI-SECURITY-002
- **Purpose**: Verify AI service cannot bypass backend security with invalid/missing tokens
- **Description**: Test security enforcement when AI service attempts backend call without proper authentication
- **Inputs**:
  - Assignment ID: `HW01`
  - User token: Invalid/expired JWT
  - AI service calls: `backend_api.get_assignment("HW01", "invalid-token")`
- **Expected Outputs**:
  - Backend returns HTTP 401 Unauthorized
  - AI service receives error response
  - AI service does NOT retrieve assignment data
  - Error propagated to frontend: "Authentication failed"
- **Test Type**: Abnormal, Whitebox, Functional, Integration

---

## Test Case 11: Qdrant Integration - Collection Creation

- **Identifier**: VECTOR-001

- **Purpose**: Verify Qdrant collection initialization with correct vector dimensions
- **Description**: Test vector store wrapper creates collection with 768-dimension vectors (Google text-embedding-004)
- **Inputs**:
  - Collection name: `CS101`
  - Vector size: 768
  - Distance metric: Cosine similarity
  - Endpoint: `POST /api/create-collection`
- **Expected Outputs**:
  - HTTP 200 response
  - Qdrant dashboard shows collection `CS101`
  - Collection config: `{vectors: {size: 768, distance: "Cosine"}}`
  - Health check confirms collection exists
- **Test Type**: Normal, Whitebox, Functional, Unit

---

## Test Case 12: Qdrant Integration - Duplicate Collection Handling

- **Identifier**: VECTOR-002
- **Purpose**: Verify system handles attempts to create already-existing collections
- **Description**: Test error handling when attempting to recreate a collection that already exists
- **Inputs**:
  - Collection name: `CS101` (already exists)
  - Endpoint: `POST /api/create-collection`
- **Expected Outputs**:
  - HTTP 409 Conflict or HTTP 200 with warning message
  - Response: `{message: "Collection already exists", collection: "CS101"}`
  - Existing collection data remains intact
  - No data loss or corruption
- **Test Type**: Boundary, Whitebox, Functional, Unit

---

## Test Case 13: CORS Configuration - Frontend API Access

- **Identifier**: CORS-001
- **Purpose**: Verify frontend at localhost:3000 can access backend at localhost:8080
- **Description**: Test CORS headers permit cross-origin requests from Next.js to Spring Boot
- **Inputs**:
  - Frontend origin: `http://localhost:3000`

    - Backend endpoint: `GET http://localhost:8080/api/users/123`
    - Browser preflight: `OPTIONS` request
- **Expected Outputs**:
    - Backend responds to OPTIONS with:
        - `Access-Control-Allow-Origin: http://localhost:3000`
        - `Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE`
        - `Access-Control-Allow-Credentials: true`
    - Actual GET request succeeds
    - No browser console errors
- **Test Type**: Normal, Blackbox, Functional, Integration

---

## Test Case 14: CORS Configuration - Unauthorized Origin Block

- **Identifier**: CORS-002
- **Purpose**: Verify backend blocks requests from non-whitelisted origins
- **Description**: Test security enforcement when request originates from unauthorized domain
- **Inputs**:
    - Frontend origin: `http://malicious-site.com`
    - Backend endpoint: `GET http://localhost:8080/api/users/123`
- **Expected Outputs**:
    - Backend rejects request
    - Browser console error: "CORS policy blocked"
    - No data returned to unauthorized origin
    - Response does NOT include `Access-Control-Allow-Origin` header for malicious domain
- **Test Type**: Abnormal, Blackbox, Functional, Integration

---

## Test Case 15: PDF Parser - Chunking Strategy

- **Identifier**: PARSER-001
- **Purpose**: Verify PDF text extraction and chunking produces correct token counts
- **Description**: Test parser splits documents into 512-token chunks with 128-token overlap
- **Inputs**:
    - File: `course_materials/chapter1.pdf` (2000 tokens total)
    - Chunk size: 512 tokens
    - Overlap: 128 tokens
- **Expected Outputs**:
    - 4 chunks generated: [(0-512), (384-896), (768-1280), (1152-1664), (1536-2000)]

    - Overlap preserves context: last 128 tokens of chunk N equal first 128 of chunk N+1
    - Metadata includes: `{chunk_index, start_token, end_token, page_number}`
    - No truncated sentences at chunk boundaries (respects paragraph breaks)
- **Test Type**: Normal, Whitebox, Functional, Unit

---

## Test Case 16: PDF Parser - Empty Document Handling

- **Identifier**: PARSER-002
- **Purpose**: Verify graceful handling of empty or corrupted PDF files
- **Description**: Test parser error handling when file contains no extractable text
- **Inputs**:
    - File: `empty.pdf` (0 pages or blank pages)
    - Endpoint: `POST /api/ingest`
- **Expected Outputs**:
    - HTTP 400 Bad Request
    - Response: `{error: "No text content found in document"}`
    - No vectors created in Qdrant
    - No partial ingestion (transaction rolled back)
- **Test Type**: Abnormal, Whitebox, Functional, Unit

---

## Test Case 17: Embedding Generation - Batch Processing

- **Identifier**: EMBED-001
- **Purpose**: Verify embeddings API batches requests efficiently (max 100 docs/call)
- **Description**: Test embedding generation respects Google API rate limits and batching
- **Inputs**:
    - 250 text chunks from ingestion
    - Embedding model: `text-embedding-004`
    - Batch size: 100
- **Expected Outputs**:
    - 3 API calls made to Google: [chunks 0-99], [100-199], [200-249]
    - Each call returns 768-dimension vectors
    - Total processing time < 60 seconds
    - All 250 embeddings stored in Qdrant
    - No rate limit errors (429 Too Many Requests)
- **Test Type**: Normal, Whitebox, Performance, Unit

---

## Test Case 18: Streaming Response - First Token Latency

- **Identifier**: PERF-001
- **Purpose**: Verify AI responses begin streaming within 3 seconds
- **Description**: Test perceived latency by measuring time to first token in SSE stream
- **Inputs**:
  - Query: "Explain merge sort algorithm"
  - Course ID: `CS101`
  - Qdrant collection: Pre-populated with 1000 vectors
- **Expected Outputs**:
  - First SSE chunk received within 3000ms
  - Subsequent chunks stream continuously
  - Total response time < 30 seconds
  - No client-side timeout errors
  - User sees progressive text rendering
- **Test Type**: Normal, Blackbox, Performance, Integration

---

## Test Case 19: Vector Search - Semantic Similarity Threshold

- **Identifier**: SEARCH-001
- **Purpose**: Verify Qdrant returns only high-relevance results (similarity > 0.7)
- **Description**: Test vector search filters out low-quality matches using cosine similarity threshold
- **Inputs**:
  - Query: "binary trees"
  - Collection: `CS101` (contains algorithms and data structures)
  - Similarity threshold: 0.7
  - Top K: 5
- **Expected Outputs**:
  - Returns 3-5 results (not forced to return 5 if insufficient matches)
  - All results have `score >= 0.7`
  - Results ranked by similarity (descending)
  - Metadata includes: `{text, score, chunk_index, page}`
- **Test Type**: Normal, Whitebox, Functional, Unit

---

## Test Case 20: Database Persistence - User Profile Update

- **Identifier**: DB-001
- **Purpose**: Verify MongoDB persists user profile changes correctly

- **Description**: Test PATCH endpoint updates user document and confirms persistence
- **Inputs**:
  - User ID: `user123`
  - Update payload: `{firstName: "John", lastName: "Doe", phone: "555-1234"}`
  - Endpoint: `PATCH /api/users/user123`
- **Expected Outputs**:
  - HTTP 200 response
  - Response body contains updated user profile
  - MongoDB document updated (verify with direct query)
  - Subsequent GET request returns new values
  - Email field unchanged (not in payload)
- **Test Type**: Normal, Blackbox, Functional, Integration

---

## Test Case 21: Concurrent User Sessions

- **Identifier**: CONCURRENCY-001
- **Purpose**: Verify system handles multiple simultaneous users without data corruption
- **Description**: Test concurrent access when 10 users query AI service simultaneously
- **Inputs**:
  - 10 concurrent HTTP requests to `POST /api/chat`
  - Each with different query and user token
  - Same course collection: `CS101`
- **Expected Outputs**:
  - All 10 requests receive correct responses
  - No cross-contamination of responses (User A doesn't get User B's answer)
  - No database deadlocks or connection pool exhaustion
  - Response times degrade gracefully (< 2x normal latency)
  - No HTTP 500 errors
- **Test Type**: Boundary, Blackbox, Performance, Integration

---

## Test Case 22: Qdrant Connection Failure

- **Identifier**: ERROR-001
- **Purpose**: Verify graceful degradation when Qdrant service is unavailable
- **Description**: Test error handling when vector database connection fails
- **Inputs**:
  - Qdrant service: Stopped/unreachable
  - User attempts: `POST /api/chat` query
- **Expected Outputs**:

    - HTTP 503 Service Unavailable
    - Response: `{error: "AI service temporarily unavailable. Please try again later."}`
    - No application crash
    - Other services (login, profile) continue functioning
    - Error logged for debugging
- **Test Type**: Abnormal, Blackbox, Functional, Integration

---

### Test Case 23: Gemini API Key Validation

- **Identifier**: CONFIG-001
- **Purpose**: Verify system detects missing or invalid Gemini API key at startup
- **Description**: Test environment configuration validation before service accepts requests
- **Inputs**:
    - `.env` file missing `GEMINI_API_KEY`
    - Start AI service: `uvicorn app.main:app`
- **Expected Outputs**:
    - Service logs warning: "GEMINI_API_KEY not configured"
    - Health check endpoint returns: `{status: "degraded", ai_enabled: false}`
    - RAG endpoints return HTTP 503: "AI features unavailable"
    - Ingestion endpoints still function (vectorization only)
- **Test Type**: Boundary, Whitebox, Functional, Unit

---

## Part III. Test Case Matrix

| Test Case ID | Normal/Ab./Boundary | Black/Whitebox | Funct/Perform | Unit/Integr |
|---|---|---|---|---|
| AUTH-001 | Normal | Blackbox | Functional | Integration |
| AUTH-002 | Normal | Blackbox | Functional | Integration |
| AUTH-003 | Abnormal | Blackbox | Functional | Integration |
| PROFILE-001 | Normal | Blackbox | Functional | Integration |
| PROFILE-002 | Abnormal | Blackbox | Functional | Integration |
| AI-INGEST-001 | Normal | Blackbox | Functional | Integration |

| Test Case ID | Normal/Ab./Boundary | Black/Whitebox | Funct/Perform | Unit/Integr |
|---|---|---|---|---|
| AI-RAG-001 | Normal | Blackbox | Functional | Integration |
| AI-RAG-002 | Boundary | Blackbox | Functional | Integration |
| AI-SECURITY-001 | Normal | Whitebox | Functional | Integration |
| AI-SECURITY-002 | Abnormal | Whitebox | Functional | Integration |
| VECTOR-001 | Normal | Whitebox | Functional | Unit |
| VECTOR-002 | Boundary | Whitebox | Functional | Unit |
| UI-I18N-001 | Normal | Blackbox | Functional | Unit |
| UI-I18N-002 | Boundary | Whitebox | Functional | Unit |
| CORS-001 | Normal | Blackbox | Functional | Integration |
| CORS-002 | Abnormal | Blackbox | Functional | Integration |
| PARSER-001 | Normal | Whitebox | Functional | Unit |
| PARSER-002 | Abnormal | Whitebox | Functional | Unit |
| EMBED-001 | Normal | Whitebox | Performance | Unit |
| PERF-001 | Normal | Blackbox | Performance | Integration |
| SEARCH-001 | Normal | Whitebox | Functional | Unit |
| DB-001 | Normal | Blackbox | Functional | Integration |
| CONCURRENCY-001 | Boundary | Blackbox | Performance | Integration |
| ERROR-001 | Abnormal | Blackbox | Functional | Integration |
| CONFIG-001 | Boundary | Whitebox | Functional | Unit |